

Diseño de Interfaces de Usuario

Práctica 5.

MiniEstudio de Diseño

Escuela de Ingeniería Informática de Segovia · Universidad de Valladolid

Contexto y objetivo

Eres el desarrollador de una pequeña herramienta de escritorio para diseñadores gráficos. La aplicación, llamada MiniEstudio, permitirá explorar composiciones de color, dibujar libremente con el ratón y animar un cursor personalizado, todo dentro de una única ventana Swing bien estructurada.

Esta práctica integra los contenidos de los cuatro primeros laboratorios de la asignatura:

- Lab 1 — Layouts de Java Swing
- Lab 2 — Dibujo con Java2D
- Lab 3 — Manejo de eventos
- Lab 4 — Clase Timer y animaciones

Parte	Contenido
Parte 1	Estructura de la ventana (Layouts)
Parte 2	Composición gráfica con Java2D
Parte 3	Interacción con eventos
Parte 4	Animación con Timer
Integración	Unificación en una sola ventana

Parte 1 — Estructura de la ventana (Layouts)

Objetivo

Construir el esqueleto visual de MiniEstudio usando los layouts aprendidos en el Lab 1.

Especificación

La ventana tendrá un tamaño fijo de **800 × 600 px** y no será redimensionable.

El JFrame raíz usa BorderLayout y contiene las siguientes regiones:

Región	Layout interno	Tamaño mínimo	Contenido
NORTH	FlowLayout	Altura 50 px	Barra de herramientas con título y botones de acción
CENTER	—	Ocupa el resto	Canvas principal de dibujo y animación
SOUTH	FlowLayout	Altura 30 px	Barra de estado: coordenadas, color activo, grosor
WEST	GridLayout (col. única)	Anchura 120 px	Paleta de color (6 celdas)

Requisito de etiquetas

Añade una JLabel identificativa con el nombre del layout usado en al menos cuatro regiones distintas, tal como se pedía en el Lab 1.

Parte 2 — Composición gráfica con Java2D

Objetivo

Dibujar una composición de estilo neoplasticista en el canvas usando paintComponent, al igual que en el Lab 2.

Paleta de color (panel WEST)

El panel lateral contendrá seis celdas de color pintadas directamente con paintComponent sobreescrito (no con setBackground). Tres de los seis colores son obligatorios:

Constante	Color RGB	Descripción
MONDRIAN_RED	(237, 28, 36)	Rojo Mondrian
MONDRIAN_BLUE	(0, 56, 168)	Azul Mondrian
MONDRIAN_YELLOW	(255, 221, 51)	Amarillo Mondrian

Los tres colores restantes son de libre elección.

Composición en el canvas

Dibuja en el canvas principal una composición inspirada en Mondrian o en Peter Keler usando `fillRect` y `drawLine`. Las líneas negras deben tener un grosor de 10 px. Las proporciones de los rectángulos deben expresarse de forma relativa al tamaño del canvas (usa porcentajes sobre `getWidth()` y `getHeight()`, no píxeles fijos), igual que el grid del Lab 2.

Anti-aliasing obligatorio

Activa el anti-aliasing en `paintComponent` con:

```
g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
    RenderingHints.VALUE_ANTIALIAS_ON);
```

Parte 3 — Interacción con eventos

Objetivo

Hacer que `MiniEstudio` responda a la interacción del usuario mediante los distintos tipos de listeners estudiados en el Lab 3.

3a — Selección de color

Al hacer clic (`mouseClicked`) sobre una celda de la paleta lateral, el color activo cambia al color de esa celda. El color activo se refleja en la barra de estado. La celda seleccionada queda visualmente destacada mediante un borde negro de 3 px.

3b — Dibujo libre

Al arrastrar el ratón (`mouseDragged`) sobre el canvas, la aplicación dibuja trazos continuos con el color activo. El grosor del trazo es el establecido en ese momento. Los trazos se acumulan: no se borran al repintar el canvas.

Pista

Para acumular trazos sin perderlos en cada repaint, guarda los segmentos dibujados en un `ArrayList` y redibújalos todos en cada llamada a `paintComponent`.

3c — Teclado

Mientras el canvas tenga el foco, el teclado controla los siguientes parámetros:

Tecla	Acción
C	Borra todos los trazos del canvas (limpia el <code>ArrayList</code> de segmentos).
+	Aumenta el grosor del trazo en 2 px (máximo 20 px). La barra de estado se actualiza.

Tecla	Acción
–	Reduce el grosor del trazo en 2 px (mínimo 2 px). La barra de estado se actualiza.

Usa MouseAdapter, MouseMotionAdapter y KeyAdapter según corresponda.

Parte 4 — Animación con Timer

Objetivo

Incorporar un sprite animado que se mueva por el canvas usando la clase Timer de Swing, según lo visto en el Lab 4.

Requisitos

Los botones Iniciar y Pausar de la barra superior arrancan y detienen la animación usando start() y stop() del Timer. Mientras la animación está activa:

1. Movimiento en dos ejes. El sprite se mueve simultáneamente en horizontal y vertical, rebotando al alcanzar cualquiera de los cuatro bordes del canvas. Los trazos que el usuario haya dibujado no se borran.
2. Animación del sprite. El sprite alterna entre al menos dos imágenes o formas con una frecuencia constante, generando la ilusión de movimiento. Puedes usar las imágenes GIF del Lab 4 o, si no dispones de ellas, un círculo que cambia de radio o color.
3. Orientación. El sprite usa una imagen (o variante) distinta según la dirección horizontal de movimiento: una para la derecha y otra para la izquierda.
4. Visualización de coordenadas. La barra de estado muestra las coordenadas actuales (x, y) del sprite, actualizadas en cada tick del Timer.
5. Control de velocidad. Un JSlider en la barra inferior permite al usuario ajustar el paso de movimiento (STEP_SIZE) entre 1 y 20 px/tick en tiempo real.
6. Rastro de movimiento. Se guardan en un ArrayList las últimas 10 posiciones del sprite. En cada repaint se dibujan como una estela de círculos de tamaño y opacidad decrecientes, de modo que se aprecie la trayectoria reciente. Para la opacidad usa AlphaComposite.

Coexistencia con el dibujo libre

El orden de pintado en paintComponent debe ser: (1) composición Mondrian de fondo, (2) trazos libres del usuario, (3) estela del sprite, (4) sprite.

Requisitos transversales

- Toda la GUI se lanza desde `SwingUtilities.invokeLater`.
- Ningún componente usa `setBounds` ni coordenadas absolutas; todo se gestiona con layouts.
- El código está organizado en clases internas con responsabilidad única: una para el canvas, una para la paleta, una para el sprite.
- Al menos cuatro regiones distintas tienen una `JLabel` identificativa del layout utilizado.
- El anti-aliasing está activado en todos los métodos `paintComponent`.

Entrega

Un único proyecto IntelliJ IDEA con las clases organizadas en paquetes separados por parte:

Paquete	Contenido
p1.layouts	Estructura de la ventana y distribución de paneles
p2.java2d	Paleta de color y composición Mondrian
p3.eventos	Listeners de ratón y teclado
p4.animacion	Sprite, Timer y estela
(raíz)	Clase Main que lanza la aplicación completa integrada